

Spring Data & Repository

Chapter 03 -- Spring Framework

Sub-Chapter 03-06

1. Spring Data Overview	JPA, MongoDB, Redis -- one unified abstraction
2. Repository Hierarchy	Repository -> CrudRepository -> JpaRepository
3. Derived Query Methods	findByName, findByAgeGreaterThan and more
4. @Query Annotation	JPQL and native SQL queries
5. Pagination and Sorting	Pageable, Page, Sort
6. @Entity and Column Basics	@Id, @GeneratedValue, @Column
7. Custom Repository Impl	Extend with your own logic

1 Spring Data Overview

One abstraction for JPA, MongoDB, Redis and more

Spring Data is an umbrella project that provides a consistent, high-level abstraction for data access regardless of the underlying store (relational DB, NoSQL, cache). Instead of writing boilerplate DAO code you define an interface and Spring generates the implementation.

Simple Example -- declaring a JPA repository:

```
// 1. Entity
@Entity
public class Product {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private double price;
    // getters / setters ...
}

// 2. Repository interface (no implementation needed)
public interface ProductRepository
    extends JpaRepository<Product, Long> {
}
```

Spring Data modules at a glance:

Module	Store	Dependency
Spring Data JPA	Relational DB via JPA/Hibernate	spring-boot-starter-data-jpa
Spring Data MongoDB	MongoDB documents	spring-boot-starter-data-mongodb
Spring Data Redis	Redis key-value store	spring-boot-starter-data-redis
Spring Data JDBC	Plain JDBC, no ORM overhead	spring-boot-starter-data-jdbc

Key Rules

- All Spring Data modules share the Repository abstraction.
- Auto-configuration in Spring Boot wires everything with minimal setup.
- Switch stores by changing the dependency and adjusting the entity model.
- Spring Data REST can expose repositories as REST endpoints automatically.

Q1. What is Spring Data?

An umbrella project providing a unified data-access abstraction over many stores (JPA, MongoDB, Redis, etc.).

Q2. Do you write the repository implementation?

No. You declare an interface extending a Spring Data interface; Spring generates the implementation at runtime.

Q3. Which dependency enables Spring Data JPA in Spring Boot?

spring-boot-starter-data-jpa, which also pulls in Hibernate as the default JPA provider.

Q4. Can Spring Data work with NoSQL stores?

Yes. Dedicated modules exist for MongoDB, Redis, Cassandra, Elasticsearch, and others.

Q5. What is Spring Data REST?

A sub-project that automatically exposes Spring Data repositories as hypermedia-driven REST endpoints.

2 Repository Hierarchy

Repository -> CrudRepository -> PagingAndSortingRepository -> JpaRepository

Spring Data defines a hierarchy of generic repository interfaces. Each level adds more operations. You choose the level that fits your needs.

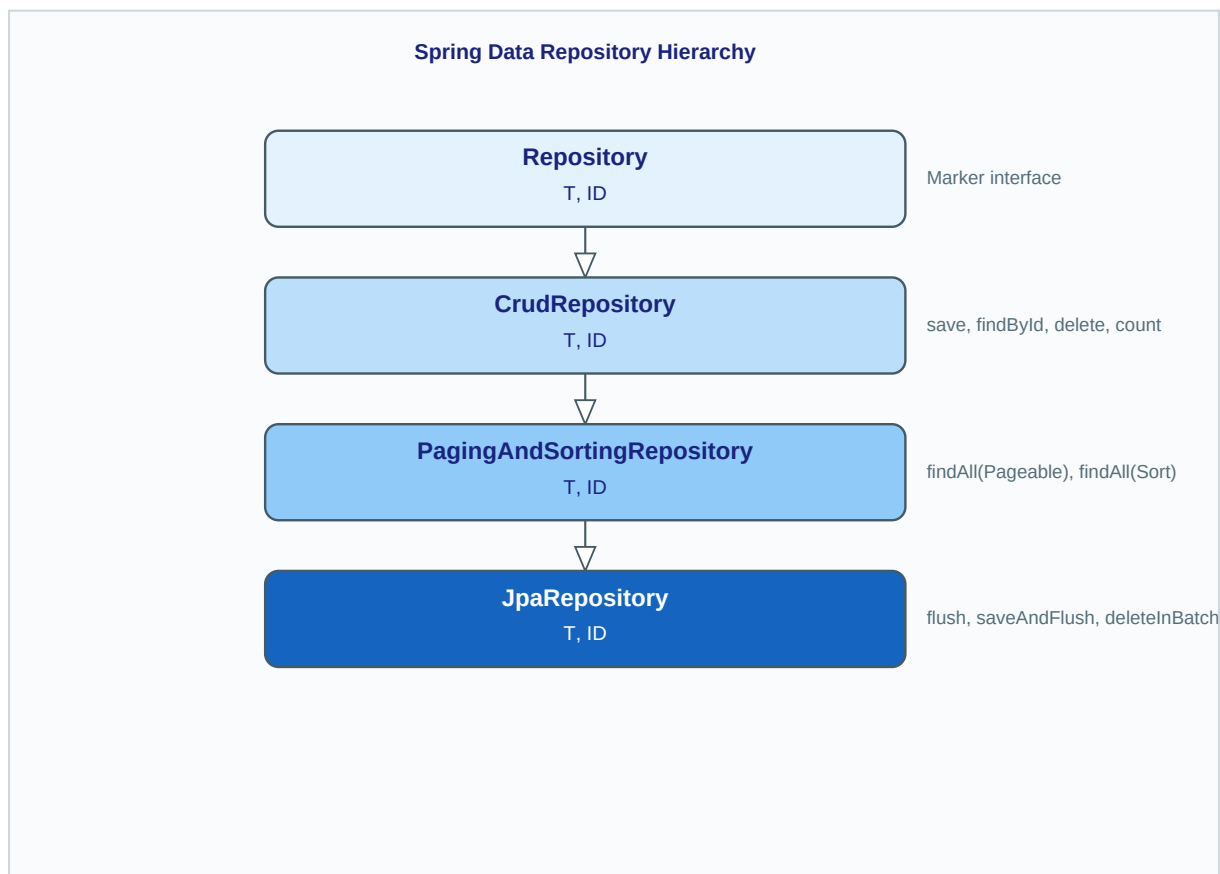


Figure: Spring Data Repository inheritance chain

Choosing the right interface:

Interface	Key Methods Added
Repository	None -- marker only
CrudRepository	save, findById, findAll, delete, count, existsById
PagingAndSortingRepository	findAll(Sort), findAll(Pageable)
JpaRepository	flush, saveAndFlush, deleteAllInBatch, getReferenceById

Key Rules

- Prefer CrudRepository for simple CRUD; use JpaRepository only when you need JPA-specific operations.
- All interfaces are generic: extend with EntityType and IdType.
- Spring creates a proxy implementation at startup -- no boilerplate code needed.

- ListCrudRepository and ListPagingAndSortingRepository return List instead of Iterable (Spring Data 3+).

Q1. What does Repository add on its own?

Nothing -- it is a marker interface. Its purpose is to let Spring detect repository interfaces during component scanning.

Q2. Which interface provides save() and delete()?

CrudRepository provides save(), saveAll(), findById(), existsById(), findAll(), deleteById(), delete(), count().

Q3. When should you use JpaRepository?

When you need JPA-specific features such as flushing the persistence context or batch deletes.

Q4. What are the two generic type parameters?

The first is the entity class (e.g., Product); the second is the type of the primary key (e.g., Long).

Q5. What does PagingAndSortingRepository add?

It adds findAll(Pageable) for pagination and findAll(Sort) for sorting results.

3 Derived Query Methods

Spring parses method names to generate queries automatically

Spring Data reads the method name and automatically generates the corresponding JPQL query. The method name follows a strict convention: Subject (find/count/exists/delete) + By + property names + optional keywords.

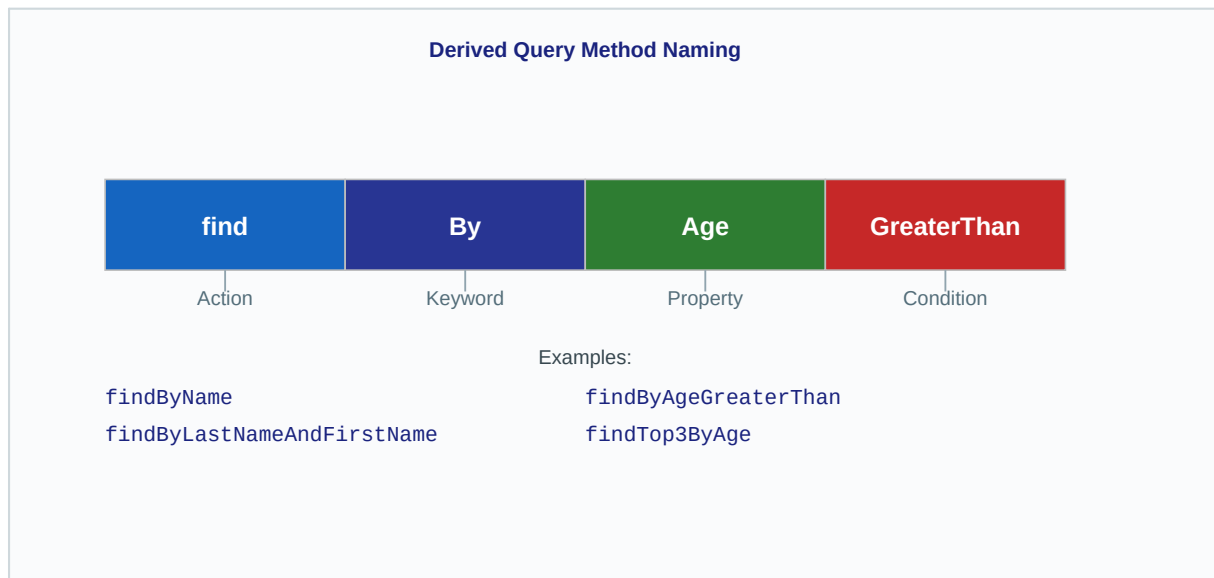


Figure: Anatomy of a derived query method name

Derived query examples:

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    // SELECT u FROM User u WHERE u.name = ?1  
    List<User> findByName(String name);  
  
    // WHERE u.age > ?1  
    List<User> findByAgeGreaterThan(int age);  
  
    // WHERE u.lastName = ?1 AND u.firstName = ?2  
    List<User> findByLastNameAndFirstName(String last, String first);  
  
    // WHERE u.email LIKE %?1%  
    List<User> findByEmailContaining(String keyword);  
  
    // Top 3 youngest users  
    List<User> findTop3ByOrderByAgeAsc();  
}
```

Key Rules

- Subject keywords: find, read, get, query, count, exists, delete, remove.
- Condition keywords: And, Or, Between, LessThan, GreaterThan, Like, Containing, StartingWith, In, IsNull, OrderBy.
- Method names become verbose for complex queries -- use @Query for those.
- Return types can be List, Optional, a single entity, long (for count), or boolean (for exists).

Q1. How does Spring know what SQL to run from a method name?

Spring Data parses the method name at startup and generates a JPQL query. No SQL is written by hand.

Q2. What does findTop5ByOrderByCreatedAtDesc do?

It returns the 5 most recently created entities, ordered by createdAt descending.

Q3. How do you find entities where a field is null?

Use findByFieldIsNull() -- Spring Data recognises IsNull and Null as synonyms.

Q4. Can derived queries return Optional?

Yes. Declaring Optional findByEmail(String email) returns an empty Optional when no result is found.

Q5. What happens if the method name refers to a non-existent property?

Spring throws a PropertyReferenceException at application startup, not at query time.

4 @Query Annotation

JPQL and native SQL for complex or hand-tuned queries

@Query lets you write the query explicitly instead of relying on method-name parsing. By default it uses JPQL (object-level language). Set nativeQuery=true for plain SQL.

JPQL query example:

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // JPQL - uses entity/field names, not table/column names
    @Query("SELECT o FROM Order o WHERE o.status = :status AND o.total > :min")
    List<Order> findByStatusAndMinTotal(
        @Param("status") String status,
        @Param("min") double min);

    // Native SQL - uses real table/column names
    @Query(value = "SELECT * FROM orders WHERE status = ?1 LIMIT 10",
        nativeQuery = true)
    List<Order> findTop10ByStatusNative(String status);

    // Modifying query (UPDATE / DELETE)
    @Modifying
    @Transactional
    @Query("UPDATE Order o SET o.status = 'CLOSED' WHERE o.id = :id")
    void closeOrder(@Param("id") Long id);
}
```

Key Rules

- JPQL references entity class names and field names, not table/column names.
- nativeQuery=true lets you write database-specific SQL.
- Modifying queries (UPDATE, DELETE) require @Modifying and @Transactional.
- Use named parameters (:name with @Param) rather than positional (?1) for readability.

Q1. What language does @Query use by default?

JPQL (Java Persistence Query Language), which operates on entity objects rather than database tables.

Q2. How do you run a plain SQL query with @Query?

Set nativeQuery = true: @Query(value = "SELECT * FROM users", nativeQuery = true).

Q3. Why is @Modifying needed?

Spring Data needs to know the query modifies data so it can clear the persistence context after execution.

Q4. What is the difference between ?1 and :name parameters?

?1 is positional (tied to argument order); :name is a named parameter used with @Param -- named is preferred.

Q5. Can @Query be used for INSERT?

Only via native queries with @Modifying. JPQL does not support INSERT statements.

5 Pagination and Sorting

Pageable, Page, and Sort -- handle large result sets

Returning millions of rows at once is impractical. Spring Data provides Pageable to request a specific page of results, Page to carry the results plus metadata, and Sort for ordering.

Pagination example:

```
@Service
public class ProductService {

    @Autowired
    private ProductRepository repo;

    public Page<Product> getPage(int pageNo, int pageSize) {
        // PageRequest is the standard Pageable implementation
        Pageable pageable = PageRequest.of(pageNo, pageSize,
            Sort.by("price").ascending());
        return repo.findAll(pageable);
    }
}

// Repository - no extra code needed
// JpaRepository already provides: Page<Product> findAll(Pageable p);
```

Useful Page methods:

Method	Returns
getContent()	List of entities on this page
getTotalElements()	Total number of records in the DB
getTotalPages()	Total number of pages
getNumber()	Current page index (0-based)
hasNext()	True if a next page exists

Key Rules

- PageRequest.of(page, size) -- page is 0-indexed.
- Pass Pageable to any repository method (derived or @Query) for pagination support.
- Sort.by(...).descending() / .ascending() creates multi-field sort chains.
- Use Slice instead of Page if you do not need the total count (faster for large tables).

Q1. What is Pageable?

An interface that carries page number, page size, and optional sort information. Pass it to repository methods.

Q2. How do you get the second page of 20 items?

PageRequest.of(1, 20) -- page index starts at 0, so index 1 is the second page.

Q3. What extra SQL does Page execute compared to Slice?

Page executes an additional COUNT query to calculate total elements and total pages; Slice does not.

Q4. How do you sort by multiple fields?

Sort.by("lastName").and(Sort.by("firstName")) or Sort.by(Order.asc("lastName"), Order.desc("age")).

Q5. Can a derived query method accept Pageable?

Yes: List findByCity(String city, Pageable pageable) returns a paged slice of results.

6 @Entity and Column Basics

@Id, @GeneratedValue, @Column -- mapping a class to a table

JPA maps a Java class to a database table using annotations. The minimum required annotations are @Entity and @Id. Additional annotations refine column names, constraints, and key generation.

Fully annotated entity example:

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String fullName;

    @Column(unique = true)
    private String email;

    @Column(name = "hire_date")
    private LocalDate hireDate;

    // Getters and setters ...
}
```

Annotation	Purpose	Key Attributes
@Entity	Marks class as a JPA entity	name (optional entity name)
@Table	Customizes the table name/schema	name, schema, uniqueConstraints
@Id	Marks the primary key field	--
@GeneratedValue	Auto-generates PK values	strategy: AUTO, IDENTITY, SEQUENCE, TABLE
@Column	Customizes a column	name, nullable, unique, length

Key Rules

- Every @Entity class must have exactly one @Id field.
- GenerationType.IDENTITY relies on DB auto-increment; SEQUENCE uses a DB sequence object.
- If @Column is omitted the field name is used as the column name.
- Fields marked insertable=false, updatable=false are managed by the DB (e.g., created_at DEFAULT NOW()).

Q1. What is the minimum to make a JPA entity?

Annotate the class with @Entity and one field with @Id. Spring Boot and Hibernate handle the rest.

Q2. What does @GeneratedValue(strategy = GenerationType.IDENTITY) mean?

The database auto-increments the primary key (e.g., MySQL AUTO_INCREMENT, PostgreSQL SERIAL).

Q3. How do you map a Java field to a differently named DB column?

Use `@Column(name = "column_name")` above the field.

Q4. Can an entity have a composite primary key?

Yes, using `@IdClass` or `@EmbeddedId` with a separate key class, though single-field keys are more common.

Q5. What happens if `nullable = false` is violated?

Hibernate throws a `ConstraintViolationException` when attempting to persist a null value into that column.

7 Custom Repository Implementation

Add hand-written logic alongside Spring-generated methods

Sometimes derived queries and `@Query` are not enough -- you may need `EntityManager` access, dynamic queries with Criteria API, or integration with a third-party library. Spring Data lets you mix custom code into any repository without losing the auto-generated methods.

Step-by-step: Custom repository fragment pattern:

```
// Step 1: Define a fragment interface
public interface ProductRepositoryCustom {
    List<Product> searchByKeyword(String keyword);
}

// Step 2: Implement the fragment
public class ProductRepositoryCustomImpl
    implements ProductRepositoryCustom {

    @PersistenceContext
    private EntityManager em;

    @Override
    public List<Product> searchByKeyword(String kw) {
        return em.createQuery(
            "SELECT p FROM Product p WHERE p.name LIKE :kw",
            Product.class)
            .setParameter("kw", "%" + kw + "%")
            .getResultList();
    }
}

// Step 3: Combine in the main repository
public interface ProductRepository
    extends JpaRepository<Product, Long>,
        ProductRepositoryCustom {
}
```

Key Rules

- The custom implementation class must be named `RepositoryInterfaceImpl` by convention.
- You can inject `EntityManager`, `JdbcTemplate`, or any Spring bean into the `Impl` class.
- Multiple custom fragments can be mixed into one repository interface.
- Spring Data merges the auto-generated and custom methods transparently at runtime.

Q1. Why would you need a custom repository?

For dynamic queries (Criteria API), full-text search, or when you need direct `EntityManager` or JDBC access.

Q2. What naming convention does Spring expect for the `Impl` class?

By default it must end with `Impl`: if the fragment interface is `MyRepoCustom`, the class is `MyRepoCustomImpl`.

Q3. Can you change the `Impl` suffix?

Yes. Set `repositoryImplementationPostfix` in `@EnableJpaRepositories` or in Spring Boot properties.

Q4. Does adding custom methods remove the generated ones?

No. The final repository bean exposes both the Spring-generated methods and your custom methods.

Q5. When should you prefer @Query over a custom implementation?

Use @Query for static, well-known JPQL/SQL queries. Reserve custom implementations for dynamic or complex logic.